Lecture 1 · Course Introduction

9/8 (화) · 슬라이드 "Lecture 1 Course Introduction" + 수업 녹취 정리

1 이 과목이 다루는 것

과목 목표

- CSED211 (Introduction to Computer Software Systems): **저수준(low-level) 소프트웨어 시스템** 입문

> **슬라이드의 네 가지 목표**
>
> 1. **Computer organization**의 기본 개념 → 컴퓨터 하드웨어 구조(architecture)에 매우 가까움
> 2. **Software가 hardware와 어떻게 함께 동작하는가** → 주어진 hardware 위에서 software를 실행하는 인터페이스
> 3. **Intel assembly language + 실습** → 하드웨어 위에서 프로그램을 직접 기술하는 수준
> 4. **System programming**으로 시스템 구성요소를 저수준에서 관리하는 개념
>
> ⇒ 네 가지 모두 "low level"을 가리킨다.

- 교재: CS:APP (Computer Systems: A Programmer's Perspective, Bryant & O'Hallaron, 3rd ed.)
- Lab: 강의와 달리 프로그램을 **직접 작성**하는 시간. assembly · system programming 실습을 TA가 지도

CS 교과과정에서의 위치

- CSED103 (프로그래밍 입문) → **CSED211** → 3·4학년 시스템 과목들 (OS, Compilers, Networks, Computer Architecture, Embedded Systems, Databases, Performance Analysis …)
- CSED211 = 이 과목들 전체의 Foundation of Computer Systems (HW · SW · networking의 기본 원리)

이 과목에서 배우는 것 → 이어지는 과목

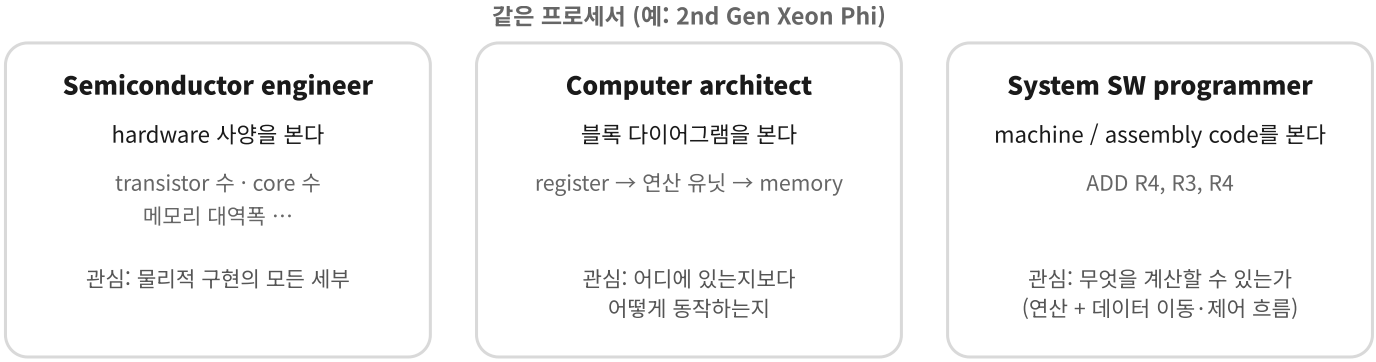
CSED211 주제	이어지는 과목
Data representation, Memory model	Databases
Network protocols	Networks
Processes, Memory management	Operating Systems
Machine code	Compilers
Arithmetic	Digital Computation
Execution model, Memory system	Computer Architecture, Embedded Systems, Performance Analysis

왜 지금 컴퓨터 시스템이 중요한가

- 양쪽에서 변화가 동시에 일어나는 중
  - 위쪽: emerging applications: 현재는 AI(neural network)가 지배적, 새로운 사용 사례가 계속 등장
  - 아래쪽: hardware 혁신: GPU, NPU 등 accelerator, GPU를 대체하려는 칩 회사들
- **시스템 분야의 역할 = 두 변화 사이의 gap을 메우는 것**. 새로운 abstraction, principle, methodology를 정의해서 새 hardware를 쓰고 새 software를 올림
- ⇒ 이 과목에서 배우는 것은 결국 abstraction을 배우는 것으로 귀결됨

2 추상화 수준 (Level of Abstraction)

같은 프로세서, 보는 사람마다 다른 모습



같은 프로세서를 보는 세 가지 관점 (슬라이드 그림 재구성)

- Xeon Phi 사양 (semiconductor engineer 관점)
  - 80억 개 이상의 transistor, self-hosted manycore processor
  - 최대 72 cores, core당 4 SMT threads, 32개의 512-bit vector register
  - 최대 384GB DDR4-2400 main memory → 최대 115GB/s
  - 최대 16GB on-package MCDRAM (3D stacked) → 최대 400GB/s
- programmer에게 이 사양은 직접적인 정보가 아님 → 모든 세부를 알 필요는 없음

손계산 1 · Xeon Phi 사양 읽기 (보충)

- 동시에 돌 수 있는 hardware thread 수: $72 \times 4 = 288$
- vector register 하나에 들어가는 32-bit int 개수: $512/32 = 16$
- vector register 32개(코어당)의 총 크기: $512 \times 32/8 = 2048$ B = 2 KB
- MCDRAM vs DDR4 대역폭 비: $400/115 \approx 3.5$배

⇒ 가까이 붙은(on-package) 작은 메모리가 훨씬 빠름 → 뒤에서 배울 **memory hierarchy**의 예고

System Software Programmer의 관점: 코드

- 슬라이드 예시: 배열 100개 원소의 합을 구하는 loop. 왼쪽 hex = machine code, 오른쪽 = assembly code (사람이 읽는 텍스트 표현) — ISA는 교육용 LC-4 (앞의 Xeon Phi 사양과는 별개 예제)

```
x9A00 CONST R5, #0
x9200 CONST R1, array
xD320 HICONST R1, array
x9464 CONST R2, sum
xD520 HICONST R2, sum
x6640 LDR R3, R1, #0
x6880 LDR R4, R2, #0
x18C4 ADD R4, R3, R4
x7880 STR R4, R2, #0
x1261 ADD R1, R1, #1
x1BA1 ADD R5, R5, #1
x2B64 CMPI R5, #100
x03F8 BRn array_sum_loop
```

- `ADD R4, R3, R4` 한 줄: "R3와 R4를 더해 R4에 저장". **무엇을** 하는지만 보여주고 data가 내부에서 어떻게 움직이는지는 숨김
- 슬라이드 18의 hex 세 개는 인코딩 오기: x1BA1은 ADD R5, R6, #1로 해독됨 (ADD R5, R5, #1 = x1B61) · x2B64는 CMPI R5, −28로 해독됨 (CMPI의 7-bit immediate는 부호 있음 — #100은 CMPI로 인코딩 불가, CMPIU R5, #100 = x2BE4가 가능한 재구성이나 원본 의도 미확인) · x03F8은 BRp로 해독됨 (BRn = x09F8). 원문은 슬라이드 그대로 유지

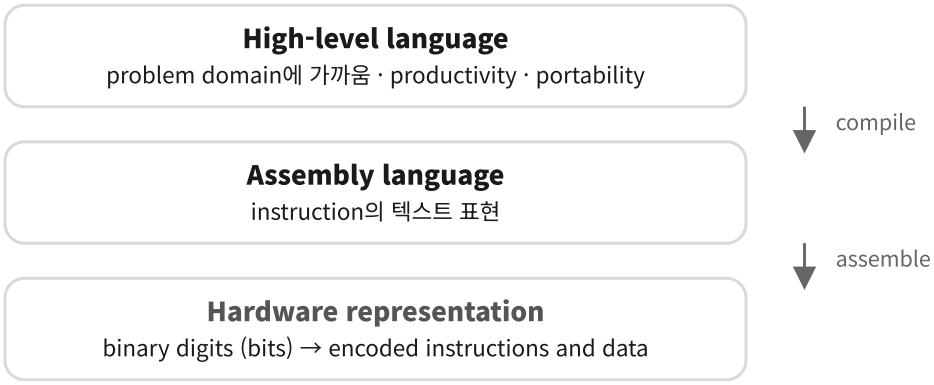
손계산 2 · Machine Code 한 줄 해독 (보충)

- 슬라이드 ISA는 Penn LC-4 (16-bit instruction, 앞 4 bit = opcode) — 13개 명령어 중 10개 인코딩이 공식 ISA와 일치 (나머지 3개는 위 슬라이드 hex 오기)
- `x18C4` = 0001 1000 1100 0100 → `ADD R4, R3, R4` (0001=ADD · 100=Rd(R4) · 011=Rs(R3) · 000 · 100=Rt(R4))
- `x6640` = 0110 0110 0100 0000 → `LDR R3, R1, #0` (0110=LDR · 011=Rd(R3) · 001=Rs(R1) · 000000=offset 0)

⇒ machine code = bit로 인코딩된 instruction, assembly = 그것의 텍스트 표현

Abstraction의 정의와 프로그램 코드의 계층

> **정의 · Abstraction**
>
> 하위 수준의 구현 세부를 숨기고 **interface만 제공**하는 것 → complexity를 다룰 수 있게 해준다. user는 "how"가 아니라 "what"에 관심을 갖는다 (hardware가 어떻게 만들어졌는지, data가 datapath에서 어떻게 움직이는지는 숨겨짐).



프로그램 코드의 세 계층: 위로 갈수록 문제에 가깝고, 아래로 갈수록 하드웨어에 가깝다

- high-level 프로그램도 결국 하위 수준으로 번역되어 hardware 위에서 실행됨 → 그래서 abstraction이 중요
- **이 과목의 관심 = 주로 assembly와 그 아래 수준**

3 앞으로 배울 주제

- 교수 코멘트: **bottom-up 방식**: 맨 아래 층부터 한 층씩 쌓아 올라감

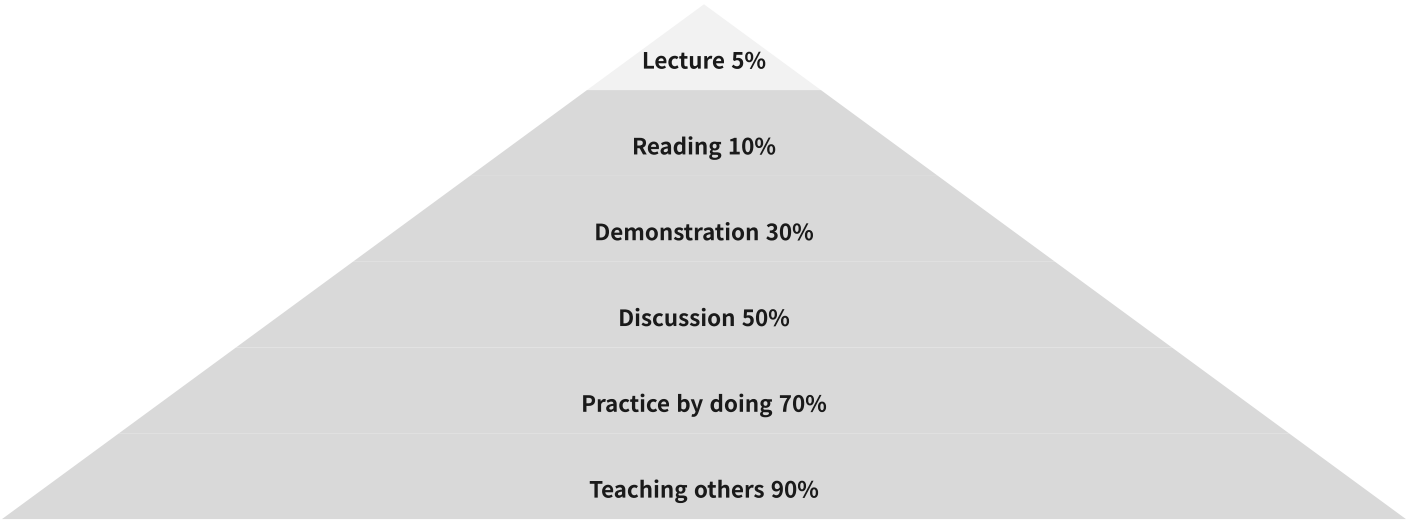
Topic 목록 (bottom-up 순서)

1. **Bits, Bytes, Integers / Floating point**: 데이터 하나가 hardware에 어떻게 저장되는가, 어떤 abstraction · guarantee가 제공되는가
2. **Machine-level programming** (Control, Procedures, Data): assembly 수준에서 실제로 hardware가 하는 일
3. **Optimizations**: 같은 결과를 내는 여러 방법 중 더 효율적인 방법
4. **Memory hierarchy, Cache memory**: 프로그램 성능에 매우 중요
5. **Linking**
6. **Exceptional control flow** (Exceptions and processes, Signals): 예: 실행 중 divide-by-zero 에러가 아래에서 어떻게 처리되는가
7. **Virtual memory, Dynamic memory allocation**: OS · 컴퓨터 구조 과목에서 반복해서 다시 나올 만큼 중요

- 읽기: Ch.1 (오늘) → Ch.2.1 (다음 강의) → Ch.2.2~2.3 (그다음 강의)

4 학습 방법

Learning Pyramid



학습 방법별 기억 유지율 (National Training Laboratory): 아래로 갈수록 능동적 — 강의 자료에 제시된 추정치 (보편적 법칙은 아니지만, 시험에서는 이 수치로 답함)

- 강의를 듣기만 하면 5% → 같은 시간에서 더 얻으려면 적극적 참여 필요
  - 동료와 토론하며 서로 이해했는지 확인 (50%), lab에서 직접 해보기 (70%)
- ⇒ 수업 · lab에서 proactive하게 참여하는 것은 교수가 아니라 **본인을 위한 것**

LLM 사용 관점

- 일반적인 개념 이해에는 웹 · AI 사용 괜찮음 (빠르고 요약과 예시가 좋음)
- LLM은 **assistant로만** → **code generator로 쓰지 말 것**
  - LLM이 생성한 코드는 GitHub 등 학습 데이터에서 온 것일 수 있음 → 그대로 쓰면 출처 코드와 일치하게 됨
  - 제출 코드는 100% 본인 책임

5 시험 포인트

- 과목 목표 네 가지: computer organization · SW와 HW의 협업 · Intel assembly · system programming
- CSED211 = OS, Compilers, Networks, Architecture 등의 foundation
- Abstraction = 하위 구현 세부를 숨기고 interface를 제공 → complexity 관리
- 같은 프로세서도 semiconductor engineer / architect / system programmer에게 다른 모습으로 보임
- 프로그램 코드 3계층: high-level language → assembly (instruction의 텍스트 표현) → machine code (bit로 인코딩된 instruction과 data)
- high-level language의 장점: problem domain에 가까움, productivity, portability
- topic 순서: bits/integers → floating point → machine-level → optimization → memory hierarchy/cache → linking → ECF → VM → malloc

6 연습문제

1-1Abstraction의 정의★

abstraction의 정의를 쓰고, 이것이 왜 필요한지 한 줄로 설명하라.

힌트 보기

섹션 2의 정의 상자 참고: "숨기는 것"과 "제공하는 것" 두 요소가 핵심.

정답 보기

개념 정리는 힌트에 있음.

① 정의를 적는다.

> "무엇을 숨기고 무엇을 보여주는가"가 뼈대

② 필요한 이유를 결과(complexity 관리)로 연결한다.

최종 답

하위 수준의 구현 세부를 숨기고 interface만 제공하는 것. 아래의 복잡한 구현(hardware 구조, data 이동 경로)을 몰라도 "무엇을" 할지만 기술할 수 있게 해 complexity를 다룰 수 있게 한다.

1-2코드 3계층 정렬★

다음 셋을 추상화 수준이 높은 순서로 나열하고 각각의 특징을 하나씩 쓰라: assembly language, machine code, high-level language.

힌트 보기

위 계층 그림에서 위쪽이 추상화가 높은 쪽. 각 계층의 한 줄 설명은 그림 아래 글머리와 같다.

정답 보기

① 가장 높은 계층부터 하나씩 특징과 짝지어 적는다.

최종 답

high-level language (problem domain에 가까움, productivity · portability) → assembly language (instruction의 textual representation) → machine code (bit로 인코딩된 instruction과 data)

1-3Xeon Phi 사양 계산★

최대 72 cores, core당 4 SMT threads인 Xeon Phi에서 동시에 실행 가능한 hardware thread 수와, 512-bit vector register 하나에 담기는 32-bit 정수의 개수를 구하라.

힌트 보기

thread 수 = core × core당 thread. 레지스터 개수 = 레지스터 비트 수 ÷ 정수 하나의 비트 수.

정답 보기

① hardware thread 수 = 72×4.

> core당 4개씩

② register의 정수 개수 = $512 \div 32$.

> 32-bit int 하나당 32 bit

최종 답 $72 \times 4 = 288$ **threads,** $512/32 = 16$**개**

1-4Machine code 해독 (보충)★★

LC-4 형식에서 ADD(register)는 `0001 ddd sss 000 ttt` 이다. `x1AC2` 를 assembly로 해독하라.

힌트 보기

hex 한 자리 = 4 bit. 16 bit로 펼친 뒤 opcode → Rd → Rs → Rt 순서로 자른다.

정답 보기

① `x1AC2` 를 이진수로 펼친다: `0001 1010 1100 0010` .

> hex → bit

② 필드대로 자른다: `0001 · 101 · 011 · 000 · 010` .

> opcode · ddd · sss · 000 · ttt

③ 각 필드를 레지스터로 번역한다: 101 = R5, 011 = R3, 010 = R2.

최종 답 **ADD R5, R3, R2**

1-5Learning Pyramid와 lab★

Learning Pyramid에서 lecturing과 practice by doing의 retention을 쓰고, 이 결과가 lab 수업 방식과 어떻게 연결되는지 설명하라.

힌트 보기

피라미드 맨 위와 아래에서 두 번째 층. lab은 "직접 작성"하는 시간이다.

정답 보기

① 두 수치를 적는다: lecturing 5%, practice by doing 70%.

② lab의 성격(직접 프로그램 작성)을 practice by doing과 연결한다.

최종 답

lecturing 5%, practice by doing 70%. lab은 직접 프로그램을 작성하는 시간이므로 듣기만 하는 것보다 retention이 훨씬 높다.